Profiling, Quantization & Constrained Devices

Week 11: CS 203 - Software Tools and Techniques for AI

Prof. Nipun Batra

IIT Gandhinagar

The Problem

Your FastAPI spam classifier works. Docker container runs.

But:

- The endpoint takes **2.5 seconds** per request
- The model file is **50 MB** — too big for a mobile app
- Your Raspberry Pi runs out of memory loading the model

Two questions:

1. **Why is it slow?** → Profiling
2. **Can we make it smaller?** → Quantization

Today's Plan

Part	Topic	Analogy
1. Profiling (40 min)	Find the bottleneck	Doctor's checkup before a diet
2. Quantization (20 min)	Make the model smaller	Packing carry-on instead of suitcase
3. Constrained Devices (10 min)	Run on phones/edge	Fitting the engine in a go-kart

Rule: Profile first. Optimize what matters. Measure again.

Part 1: Profiling — The Checkup

Before any diet, visit the doctor first.

"My Code is Slow" is Not Useful

That's like saying "I spent too much money."

You need the **itemized receipt**:

> "80% of your time is spent in one function."

A **profiler** gives you that receipt.

```
Profile → Find bottleneck → Optimize → Profile again
```

> *"Premature optimization is the root of all evil."* — Donald Knuth

Without profiling, you'll optimize the wrong thing.

Four Profiling Tools (Simplest → Most Detailed)

Tool	What It Tells You	When to Use
`time.time()`	Total runtime	Quick sanity check
`%%timeit`	Average over many runs	Comparing two approaches
`cProfile`	Time per **function**	Finding which function is slow
`line_profiler`	Time per **line**	Finding which line is slow

Think of a restaurant: `time.time()` tells you the meal took 90 minutes. `cProfile` tells you 60 minutes was waiting for the main course. `line_profiler` tells you the chef spent 45 minutes chopping onions.

Level 1: The Stopwatch

```
import time

start = time.time()
model.predict(X_test)
elapsed = time.time() - start
print(f"Prediction took {elapsed:.3f} seconds")
```

Problem: Noisy. One run isn't reliable.

Better: Average many runs.

```
import timeit
t = timeit.timeit(lambda: model.predict(X_test), number=100)
print(f"Average: {t/100*1000:.1f} ms per prediction")
```

In Jupyter: `%%timeit model.predict(X_test)`

Level 2: cProfile — The Function Receipt

```
import cProfile

def slow_endpoint(text):
    import pandas as pd
    data = pd.read_csv("training_data.csv")   # 50 MB file!
    model = joblib.load("model.pkl")            # reload every time!
    return model.predict([text])

cProfile.run('slow_endpoint("hello")')
```

Output:

```
   ncalls  tottime  function
        1    1.800  read_csv
        1    0.600  load
        1    0.001  predict
```

The bottleneck is not prediction — it's loading data and model on every call!

The Fix: Load Once

```python
# BAD — loads on every request (2.4 seconds)
@app.post("/predict")
def predict(msg):
    data = pd.read_csv("data.csv")      # 1.8s
    model = joblib.load("model.pkl")     # 0.6s
    return model.predict([msg.text])     # 0.001s

# GOOD — load once at startup (0.001 seconds)
data = pd.read_csv("data.csv")          # runs once
model = joblib.load("model.pkl")        # runs once

@app.post("/predict")
def predict(msg):
    return model.predict([msg.text])     # only this runs per request
```

250x speedup from moving two lines. That's what profiling finds.

Level 3: line_profiler — The Line Receipt

When you know which function is slow, find which line:

```
# Install: pip install line_profiler
# Add @profile decorator, run with: kernprof -l -v script.py

@profile
def predict_batch(texts):
    results = []
    for text in texts:                        # Line 1
        vec = vectorizer.transform([text]) # Line 2: 0.5ms each
        pred = model.predict(vec)           # Line 3: 0.1ms each
        results.append(pred[0])             # Line 4: 0.001ms
    return results
```

Finding: Line 2 is 5x slower than Line 3.

Fix: Vectorize the whole batch at once:

```
vecs = vectorizer.transform(texts)  # one call, not 1000
preds = model.predict(vecs)         # one call, not 1000
```

Memory Profiling

Sometimes the bottleneck isn't time — it's **memory**.

```
# pip install memory_profiler
# Run with: python -m memory_profiler script.py

@profile
def load_data():
    df = pd.read_csv("big_data.csv")    # +500 MB
    X = df.values                        # +500 MB (copy!)
    X_scaled = scaler.transform(X)       # +500 MB (another copy!)
    return X_scaled
```

1.5 GB for a 500 MB CSV! Every operation creates a copy.

Fix: Process in chunks, or use `dtype` to reduce precision:

```
df = pd.read_csv("big_data.csv", dtype="float32")  # half the memory
```

Profiling: The Cheat Sheet

```
1. "Is my code slow?"
   → time.time() or %%timeit

2. "Which function is slow?"
   → cProfile.run('my_function()')

3. "Which line in that function is slow?"
   → @profile + kernprof -l -v script.py

4. "Is it using too much memory?"
   → @profile + python -m memory_profiler script.py

5. "Does profiling slow down my code?"
   → YES! It's like an X-ray: turn on, find the problem,
     fix it, turn off. Never leave it on in production.
```

Demo: `python profiling_demo.py`

Part 2: Quantization — The Diet

The single highest-ROI optimization you can do.

What is Quantization?

Analogy: Packing for a flight.

FP32 = bringing your entire 30 kg suitcase. An outfit for every possible weather condition (high precision).

INT8 = a 7 kg carry-on. You round off to "hot weather" or "cold weather." You lose a tiny bit of precision, but you fit on the budget airline (constrained device) and move much faster.

Format	Bytes per number	Precision
FP32	4 bytes	Very high
FP16	2 bytes	High
INT8	1 byte	Moderate
INT4	0.5 bytes	Low

The Grocery Store Analogy

Instead of adding up:

```
₹4.99281 + ₹3.00192 + ₹7.49837 = ₹15.49310    (FP32: precise)
```

Just round and add:

```
₹5 + ₹3 + ₹7 = ₹15                            (INT8: fast)
```

The total is close enough. And the mental math is *way* faster.

That's quantization. Use fewer bits per number. Smaller model, faster inference, nearly the same accuracy.

Model Size by Format

Format	Bytes/param	100M params	7B params (LLaMA)
FP32	4	400 MB	**28 GB**
FP16	2	200 MB	14 GB
INT8	1	100 MB	7 GB
INT4	0.5	50 MB	**3.5 GB**

LLaMA-7B in FP32: needs a ₹8 lakh GPU with 32+ GB VRAM.
LLaMA-7B in INT4: **fits on your laptop with 4 GB free.**

Same model. Same knowledge. Just stored more efficiently.

ONNX — The PDF of Machine Learning

Analogy: You wrote a document in Google Docs. Your friend uses Word. Your client uses Pages. Export as **PDF** — everyone can read it.

ONNX does the same for models. Train in sklearn/PyTorch, run *anywhere*.

```
# Convert sklearn model to ONNX
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

initial_type = [('input', FloatTensorType([None, n_features]))]
onnx_model = convert_sklearn(clf, initial_types=initial_type)

# Run without sklearn!
import onnxruntime as ort
session = ort.InferenceSession("model.onnx")
result = session.run(None, {"input": X_test.astype("float32")})
```

Why? ONNX Runtime auto-optimizes, runs 2-5x faster, and works on mobile/browser.

Demo: Quantize an sklearn Model

```python
from sklearn.neural_network import MLPClassifier
from skl2onnx import convert_sklearn
from onnxruntime.quantization import quantize_dynamic, QuantType

# Train a deliberately large MLP
clf = MLPClassifier(hidden_layer_sizes=(500, 500, 500))
clf.fit(X_train, y_train)

# Convert to ONNX (FP32)
onnx_model = convert_sklearn(clf, initial_types=initial_type)
with open("model_fp32.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

# Quantize to INT8
quantize_dynamic("model_fp32.onnx", "model_int8.onnx",
                 weight_type=QuantType.QUInt8)
```

```
Original FP32:  3.82 MB
Quantized INT8: 0.98 MB  → 74% smaller!
Accuracy drop:  < 0.5%
```

Demo: `python quantization_demo.py`

The LLaMA Story

In 2023, Meta released **LLaMA-7B** — powerful but 28 GB in FP32.

Then Georgi Gerganov built **llama.cpp** with 4-bit quantization:

	Before (FP32)	After (INT4)
Size	28 GB	3.5 GB
Hardware	₹8 lakh GPU	MacBook Air
Cost	Cloud GPU rental	Free (your laptop)
Privacy	Data sent to cloud	Everything stays local

Suddenly, a powerful LLM runs on a phone. On a Raspberry Pi.

Quantization didn't just save money — it democratized AI.

Part 3: Constrained Devices

Fitting the engine in a go-kart.

Deployment Targets Have Budgets

Target	Size Budget	Speed Budget	Framework
Cloud API	Unlimited	< 100 ms	PyTorch, ONNX
Mobile app	< 50 MB	< 50 ms	TF Lite, ONNX
IoT / Edge	< 10 MB	CPU only	TF Lite, ONNX
Web browser	< 5 MB	JS only	ONNX.js, TF.js
LLM on laptop	< 16 GB	Usable	llama.cpp (INT4)

Training happens once. Inference happens millions of times.

Every millisecond saved, every megabyte trimmed — multiplied by every user, every request, every day.

Real Examples on Your Phone

App Feature	Model	Runs On
Keyboard prediction	Small LSTM	On-device
Face unlock	MobileFaceNet (< 5 MB)	On-device
Camera HDR	Tiny CNN	On-device
"Hey Siri" / "OK Google"	Wake word detector (< 1 MB)	On-device
Google Translate (offline)	Quantized transformer	On-device

None of these send data to a server. They're small enough to run locally, which means:

- Works offline
- No latency
- User data stays private

The Optimization Pipeline

```
Large Model (FP32, 400 MB, 90 ms)
    │
    ├── Export to ONNX   → framework-independent
    ├── Quantize (INT8)  → 4x smaller, faster on CPU
    ├── Prune (optional) → remove near-zero weights
    └── Distill (optional) → train smaller student model
    │
Small Model (INT8, 60 MB, 8 ms) — 97% accuracy
```

Start simple: ONNX export + INT8 quantization gets you most of the way.

Only add pruning/distillation if you need to go further.

FAQ

Q: "If INT8 is 4x smaller and just as accurate, why isn't everything quantized?"

→ Training needs FP32 (tiny gradients need decimal precision). Quantization is an extra step you do at the end, and it requires testing.

Q: "I quantized my model but it's actually SLOWER. Why?"

→ If your CPU lacks specialized INT8 instructions, it converts INT8 → FP32 → compute → INT8 on every operation. The model is smaller on disk but slower to run.

Q: "Why ONNX instead of just .pkl files?"

→ Pickle is Python-only. ONNX runs on iPhone (Swift), Android (Kotlin), browser (JavaScript), and any language with ONNX Runtime. It's the PDF of ML.

Practical Workflow

```
1. PROFILE first
   → time.time(), cProfile, line_profiler
   → Find the actual bottleneck (it's never where you think)

2. Fix the OBVIOUS things
   → Load model once, not per-request
   → Vectorize loops
   → Use float32 instead of float64

3. QUANTIZE if the model is too big
   → Export to ONNX → quantize_dynamic → INT8
   → Check accuracy hasn't dropped

4. MEASURE everything
   → Before/after size, latency, accuracy
   → Stop when you hit your target
```

Key Takeaways

1. **Profile first, optimize the bottleneck** — don't guess, measure
2. `cProfile` tells you which function is slow; `line_profiler` tells you which line
3. **Load models/data once at startup** — the #1 speedup for web apps
4. **Quantization = highest ROI** — 4x smaller, ~0% accuracy loss
5. **ONNX = the PDF of ML** — train in Python, run anywhere
6. **Constrained devices have budgets** — mobile < 50 MB, IoT < 10 MB, browser < 5 MB
7. **Start simple, measure, stop when good enough**